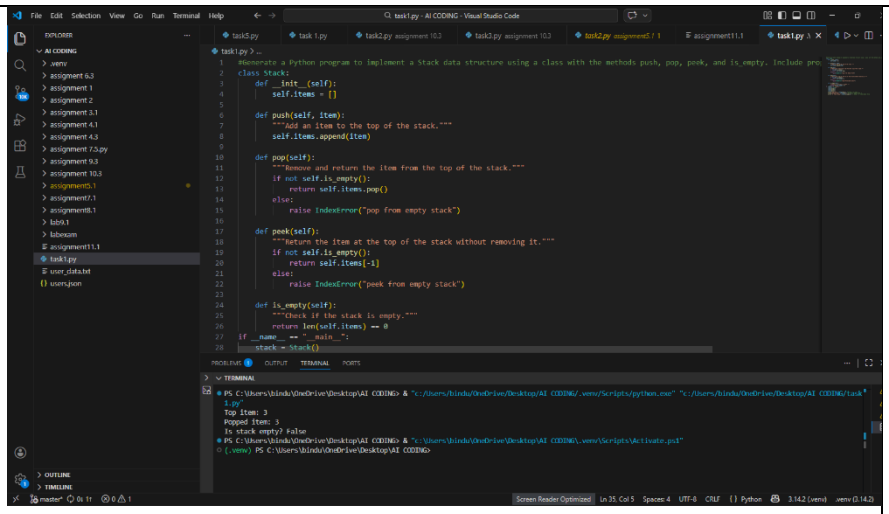
SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		**Assignment Type: Lab**	**Academic Year:**2025-2026
Course Coordinator Name	Dr. Rishabh Mittal		
CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
Name:	**v.sushmitha**	**Hallticket**	**2303A54055**
Duration	2 Hours	**Batch**	**47_b**
Assignment Number: 11.1			

Q.No.	Question	*Expected Time to complete*
1	**Lab 11 – Data Structures with AI: Implementing Fundamental Structures** **Lab Objectives** • Use AI to assist in designing and implementing fundamental data structures in Python. • Learn how to prompt AI for structure creation, optimization, and documentation. • Improve understanding of Lists, Stacks, Queues, Linked Lists, Trees, Graphs, and Hash Tables. • Enhance code quality with AI-generated comments and performance suggestions.	Week6 - Monday
	Task Description #1 – Stack Implementation **Task**: Use AI to generate a Stack class with push, pop, peek, and is_empty methods. **Prompt Used** Generate a Python program to implement a **Stack data structure using a class** with push, pop, peek, and is_empty methods. Include comments and docstrings.	



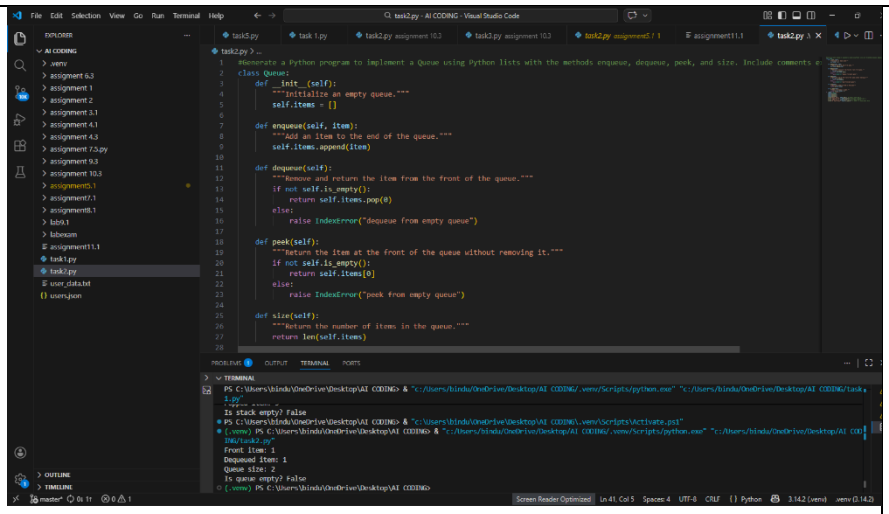
Sample Input
push 10
push 20
push 30

Sample output
Top item: 3
Popped item: 3
Is stack empty? False

Explanation
Stack follows the lifo (Last In First Out) principle. Elements are inserted using push and removed using pop. The peek method shows the top element without removing it.

Task Description #2 – Queue Implementation
Task: Use AI to implement a Queue using Python lists.

Prompt Used
Generate a Python program to implement a **Queue using Python list** with enqueue, dequeue, peek, and size methods.



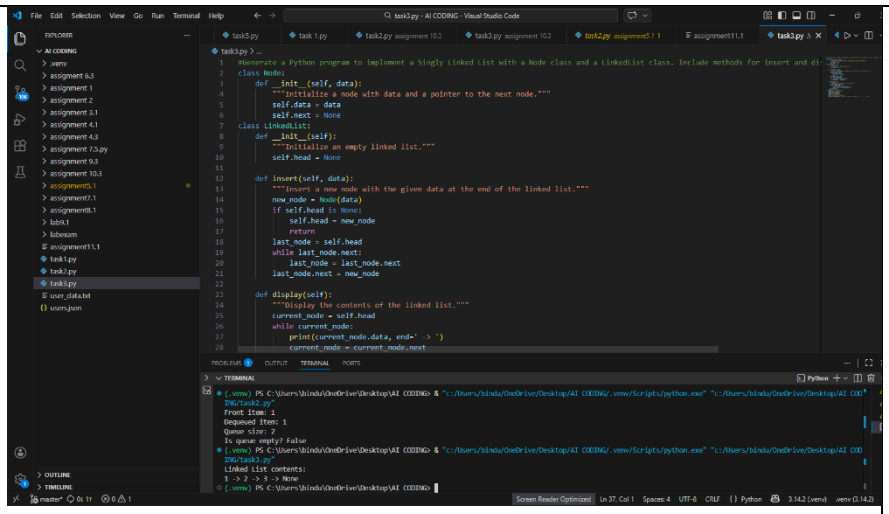
Sample Input
enqueue 1
enqueue 2
enqueue 3
dequeue
Sample Output
1
Explanation

Queue works on FIFO (First In First Out) principle where the first element inserted is the first element removed

---

Task Description #3 – Linked List
Task: Use AI to generate a Singly Linked List with insert and display methods.

Prompt Used
Generate a Python program to implement a **Singly Linked List** with node creation, insertion, and display functions.



Sample Output

5 -> 10 -> 15

Explanation

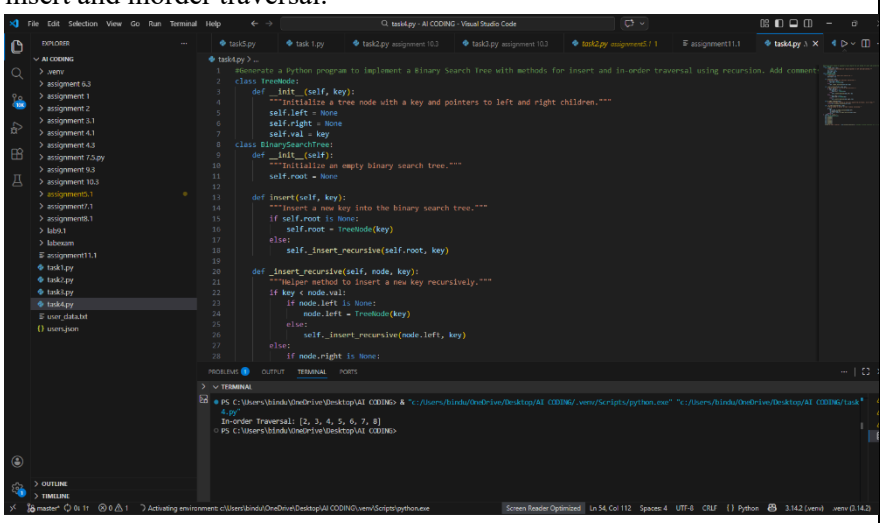
A linked list stores elements in nodes where each node contains data and a reference to the next node.

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Prompt Used

Generate a Python program to implement a Binary Search Tree with insert and inorder traversal.



Sample Output:

(2,3,4,5,6,7,8)

Explanation

BST stores values such that **left child < parent < right child**, allowing efficient searching and sorting.

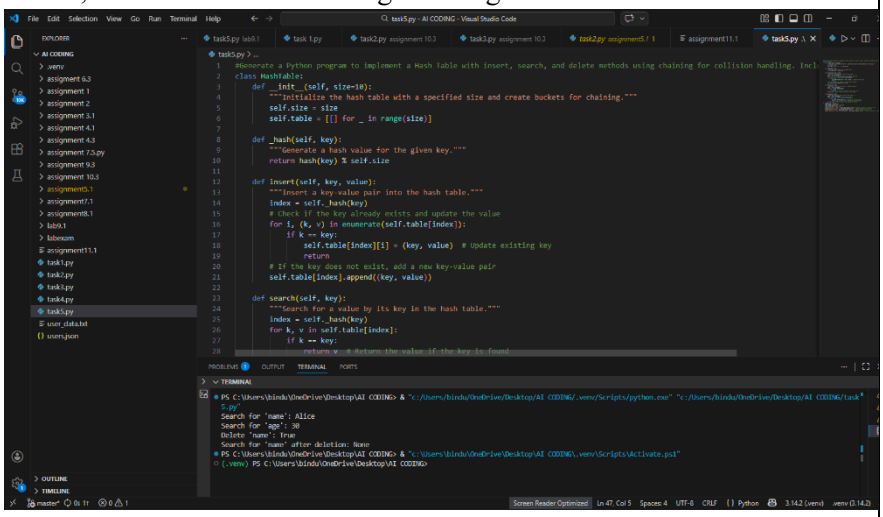
.

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Prompt Used

Generate a Python program to implement a Hash Table with insert, search, and delete methods using chaining.



SAMPLE INPUT:

Output: Search for 'name': Alice

Output: Search for 'age': 30

Output: Delete 'name': True

Output: Search for 'name' after deletion: None

SAMPLE OUTPUT:

Search for 'name': Alice

Search for 'age': 30

Delete 'name': True

Search for 'name' after deletion: None

Explanation

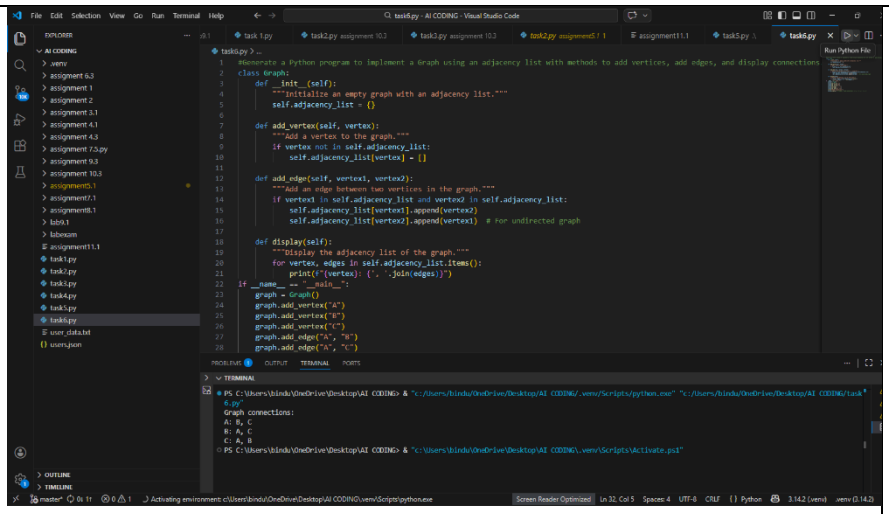
A hash table stores data using a hash function to convert keys into indexes, allowing very fast searching.

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Prompt Used

Generate a Python program to implement a Graph using adjacency list.



SAMPLE INPUT:

Output: Graph connections: A: B, C B: A, C C: A, B

SAMPLE OUTPUT:

Graph connections:
A: B, C
B: A, C
C: A, B

Explanation

Graph represents connections between nodes, commonly used for networks and route planning.

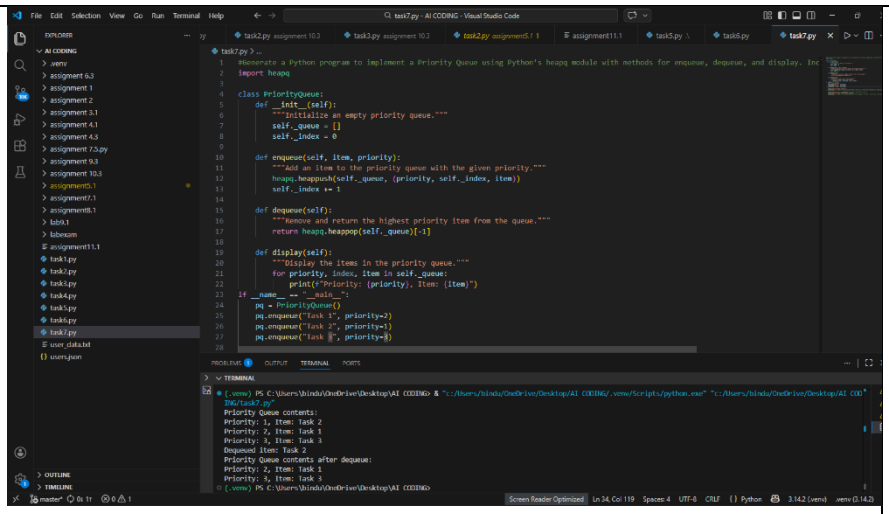
.

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

Prompt Used

Generate a Python program to implement a Priority Queue using heapq module.



Sample Input :

Output: Priority Queue contents: Priority: 1, Item: Task 2 Priority: 2, Item: Task 1 Priority: 3, Item: Task 3

Output: Priority Queue contents after dequeue: Priority: 2, Item: Task 1 Priority: 3, Item: Task 3

Output: Dequeued item: Task 2

Expected Output:

Priority Queue contents:
Priority: 1, Item: Task 2
Priority: 2, Item: Task 1
Priority: 3, Item: Task 3
Dequeued item: Task 2
Priority Queue contents after dequeue:
Priority: 2, Item: Task 1
Priority: 3, Item: Task 3

Explanation

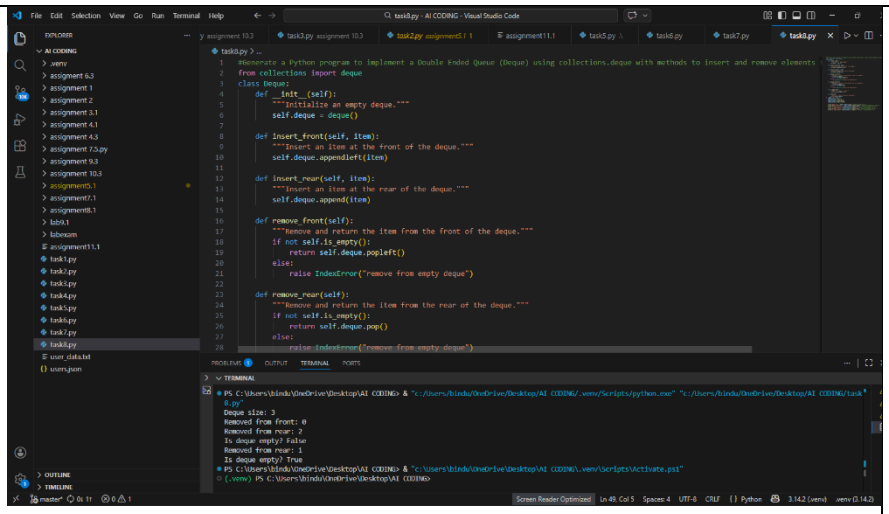
Priority queue removes elements based **on priority instead of insertion order.**

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using collections.deque.

Prompt Used

Generate a Python program to implement a Deque using collections.deque.



Sample Input :

Output: Deque size: 3

Output: Removed from front: 0

Output: Removed from rear: 2

Output: Is deque empty? False

Output: Removed from rear: 1

Output: Is deque empty? True

Expected Output:

Deque size: 3

Removed from front: 0

Removed from rear: 2

Is deque empty? False

Removed from rear: 1

Is deque empty? True

Task 9 – Campus Resource Management System

Feature → Data Structure → Justification

Student Attendance Tracking → **Deque**

Students enter and exit campus from both sides. Deque allows insertion and removal from both ends.

Event Registration → **Hash Table**

Provides fast search and deletion using student ID.

Library Book Borrowing → **BST**

Allows efficient searching and sorting of books.

Bus Scheduling → **Graph**

Represents connections between bus stops.

Cafeteria Orders → **Queue**

Orders are served in the order students arrive.

Task 10 – Smart E-Commerce Platform
Feature → Data Structure → Justification
Shopping Cart → **Linked List**
Products can be dynamically added or removed.
Order Processing → **Queue**
Orders processed in FIFO order.
Top Selling Products → **Priority Queue**
Products ranked based on sales count.
Product Search → **Hash Table**
Fast lookup using product ID.
Delivery Route Planning → **Graph**
Represents delivery routes between locations.

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure
Scenario:
Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
    - Stack
    - Queue
    - Priority Queue
    - Linked List
    - Binary Search Tree (BST)
    - Graph
    - Hash Table
    - Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

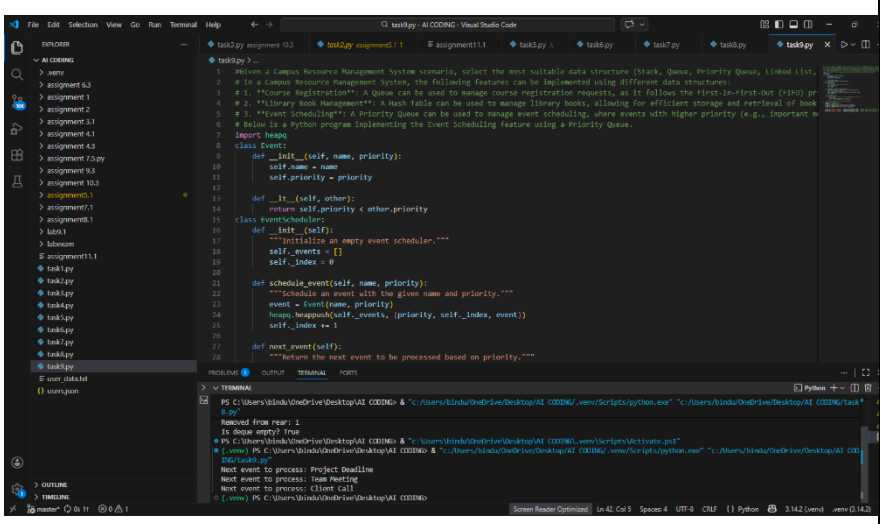
•

Sample Input :

Output: Pending Orders: Alice - Sandwich Bob - Salad Charlie - Pizza
Output: Order served: Alice - Sandwich
Output: Order served: Bob - Salad
Output: Pending Orders: Charlie - Pizza
Output: Order served: Charlie - Pizza
Output: No orders to serve.

Expected Output:

Bob - Salad
Charlie - Pizza
Order served: Alice - Sandwich
Order served: Bob - Salad
Pending Orders:
Charlie - Pizza
Order served: Charlie - Pizza
No orders to serve.



Explanation : In this task, different features of a **Campus Resource Management System** were analyzed to choose the most suitable data structure for each function. Each feature such as attendance tracking, event registration, and cafeteria orders requires a structure that handles data efficiently. For example, queues manage orders in arrival order, hash tables provide fast searching, and graphs represent route **connections**. Selecting the correct data structure helps the system work faster and manage campus resources effectively.

Task Description #10: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

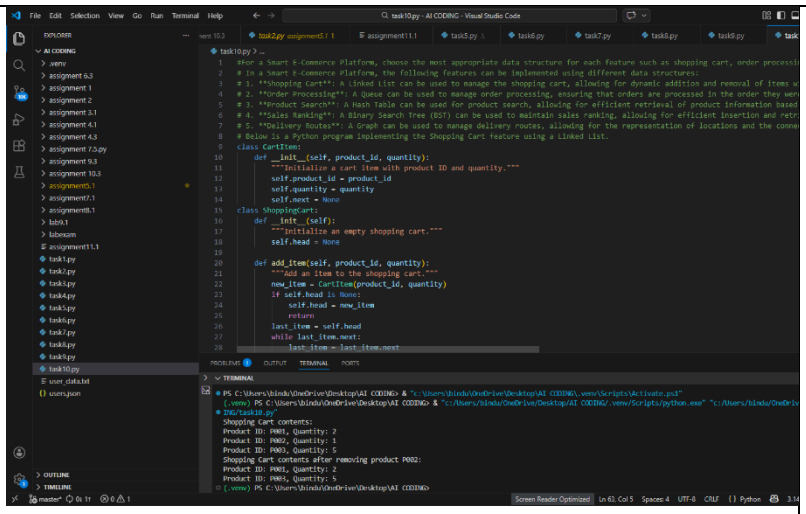
1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
  - Stack
  - Queue
  - Priority Queue
  - Linked List
  - Binary Search Tree (BST)
  - Graph
  - Hash Table
  - Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.



Sample Input :

Output: Shopping Cart contents: Product ID: P001, Quantity: 2 Product ID: P002, Quantity: 1 Product ID: P003, Quantity: 5

Output: Shopping Cart contents after removing product P002: Product ID: P001, Quantity: 2 Product ID: P003, Quantity: 5

Expected Output:

Shopping Cart contents:

Product ID: P001, Quantity: 2

Product ID: P002, Quantity: 1

Product ID: P003, Quantity: 5

Shopping Cart contents after removing product P002:

Product ID: P001, Quantity: 2

Product ID: P003, Quantity: 5

Explanation – Task 10

In this task, different features of a Smart E-Commerce Platform were analyzed to choose the most suitable data structure for each operation. Each feature such as shopping cart management, order processing, product search, and delivery routing requires an efficient way to store and manage data. By using appropriate data structures like linked lists, queues, priority queues, hash tables, and graphs, the system can perform operations faster and handle large amounts of data efficiently.